

Real-Time Insider Threat Detection Using NoSQL Analysis of the NDB Rs. 13.2 Billion Fraud

A Comparative Study of Document-Oriented vs. Relational Databases
for Event Correlation in Banking Fraud Prevention

BSc (Hons) Computer Science

Decision Analytics
Level 6

Lakshan N. Kumararathna

Student ID: CB013366

Batch: COMK23A1

Module Lecture: Dr. Rajitha Navarathne

Submission Date: May 22, 2026

Contents

1	Introduction and Problem Context	1
1.1	Background of Financial Fraud in Banking Systems	1
1.2	Case Study: NDB Bank Fraud Incident in Sri Lanka	1
1.3	Problem Statement	2
1.4	Objectives	2
1.5	Question	2
1.6	Chapter Summary	3
2	Root Cause Analysis of System Failure	4
2.1	Insider Threat Characteristics	4
2.2	Operational Monitoring Gaps	4
2.2.1	Weekend and After Hours Monitoring Limitations	5
2.2.2	Delayed Audit Procedures	5
2.2.3	Lack of Event Correlation	5
2.3	High Volume Banking Environment Challenges	5
2.4	Regulatory Considerations and Rapid Response Requirements	6
2.5	Chapter Summary	7
3	Limitations of Traditional SQL Systems	8
3.1	Schema Rigidity	8
3.2	Join Complexity and Query Performance	8
3.3	Batch Processing Limitations	9
3.4	Development Complexity	10
3.5	Chapter Summary	10
4	NoSQL as a Proposed Solution	11
4.1	Characteristics of NoSQL Databases	11
4.2	Why MongoDB Was Selected	11
4.3	Benefits for Fraud Detection Systems	12
4.4	Chapter Summary	13
5	System Design and Implementation	14
5.1	Overall System Architecture	14
5.1.1	Architectural Overview	14
5.1.2	World Engine	15
5.1.3	Rules Engine	16
5.1.4	Monitoring Dashboard	16
5.2	Dataset and Simulation Environment	16
5.2.1	Employee Profile Generation	16
5.2.2	Customer Dataset Generation	18
5.2.3	Historical Transaction Generation	18
5.2.4	Fraud Scenario Injection	19
5.3	Detection Rule Engine	20
5.3.1	After-Hours Activity Detection	20
5.3.2	Credential Abuse Detection	20
5.3.3	Suspicious Beneficiary Detection	21
5.3.4	Ghost Account Detection	22

5.3.5	Composite Risk Score Model	22
5.4	MongoDB Schema Design	23
5.4.1	employee_actions Collection	23
5.4.2	employees Collection	24
5.4.3	customers Collection	25
5.4.4	Indexing Strategy	26
5.5	Real-Time Traffic Generator	26
5.6	SOC Dashboard Design	27
5.7	Attack Simulation Module	28
5.8	Chapter Summary	29
6	System Evaluation	30
6.1	Experimental Setup	30
6.2	Detection Performance	31
6.3	System Latency	31
6.4	Scalability Analysis	32
6.5	Chapter Summary	33
7	Limitations	34
7.1	Consistency Trade-offs	34
7.2	Storage Overhead	34
7.3	Security Concerns	34
7.4	Operational Challenges	35
7.5	Future Improvements	35
7.6	Chapter Summary	35
8	Conclusion	36
8.1	Summary of Findings	36
8.2	Benefits for Banking Environments	36
8.3	Final Reflection	36
	References	37

List of Figures

2.1	Typical operational workflow	5
2.2	Signal versus noise problem in banking environments	6
3.1	General process sequence	9
5.1	Overall architecture of the proposed threat detection system.	14
5.2	MongoDB Compass	23
5.3	MongoDB Compass employee action collection	24
5.4	MongoDB Compass employee collection	25
5.5	MongoDB Compass customer collection	25
5.6	SOC Dashboard	27
5.7	Attacker simulation Dashboard	28
6.1	Successful fraud detection	31
6.2	Stages contribute to total response timen	32

List of Tables

6.1	Experimental environment	30
6.2	Detection results	31
6.3	Observed response times	32
6.4	Scalability observations	32

List of Code Snippets

3.1	Adding new attributes in SQL	8
3.2	Example SQL fraud analysis query	9
3.3	Typical SQL data conversion process (python)	10
4.1	Adding new information to a document	11
4.2	Employee activity document	12
4.3	MongoDB compound index	12
4.4	Searching suspicious employee activity	12
5.1	Employee profile generation	15
5.2	Injected employee records	15
5.3	Rules engine entry function	16
5.4	Retrieving recent employee actions	16
5.5	Employee profile generation	17
5.6	Mongosh Insert operation example	17
5.7	Injected employee identities	17
5.8	Customer profile generation	18
5.9	Mongosh retrieval example for not verified	18
5.10	Ghost account injection	18
5.11	Historical activity generation	19
5.12	Mongosh query example	19
5.13	Business hour generation	19
5.14	Initial risk score generation	20
5.15	After-hours activity detection	20
5.16	Credential abuse detection	20
5.17	Mongosh detection query example	21
5.18	Credential abuse identification	21
5.19	Suspicious beneficiary detection	21
5.20	Ghost account detection	22
5.21	Mongosh lookup example	22
5.22	Composite risk calculation	22
5.23	Alert threshold evaluation	22
5.24	employee_actions document structure	23
5.25	employees document structure	24
5.26	customers document structure	25
5.27	MongoDB indexing strategy	26
5.28	Equivalent Mongosh example	26
5.29	Mongosh delete example	26
5.30	Live activity generation	27
5.31	Retrieving recent activities	27
5.32	Fraud scenario injection	28
5.33	Mongosh update example	29
7.1	Repeated employee information in documents	34
7.2	Dashboard refresh mechanism	35

Chapter 1

Introduction and Problem Context

1.1 Background of Financial Fraud in Banking Systems

Banking systems have changed rapidly over the past few years because of digital technology. Online banking, mobile banking applications, digital payments, and real-time transaction systems have made financial services faster and easier for customers. People can now transfer money, apply for loans, and perform banking activities from almost anywhere. Although these technologies improve convenience, they also create new security challenges. As more banking activities become digital, the number of opportunities for fraud also increases. Traditionally, banking fraud was often linked to external attacks such as phishing, malware, or stolen customer information. Insider threats, another important problem has become increasingly noticeable in recent years. Insider threats occur when individuals who already have authorized access to systems misuse their privileges for harmful activities. These individuals may include employees, contractors, or third-party partners who have access to internal systems and sensitive information (Uddin, 2026). Unlike external attackers, insiders already understand how internal systems operate. They may know security procedures, approval processes, and system weaknesses. Because of this, their activities often appear similar to normal work activities and become difficult to identify. Banks also generate large amounts of information every day. Customer transactions, login activities, employee records, audit logs, and system events continuously produce data. Hidden within these large datasets may be signs of suspicious behavior. However, identifying abnormal activities becomes difficult because a single event may appear harmless when viewed independently. For example, an employee logging into the system at night may not appear suspicious by itself. However, if the same employee later approves an unusual transaction and transfers funds to an unknown account, the sequence of events may indicate fraudulent behavior. Therefore, modern fraud detection systems increasingly require real-time monitoring and behavioral analysis rather than relying only on delayed auditing methods (Sirawongphatsara et al., 2023).

1.2 Case Study: NDB Bank Fraud Incident in Sri Lanka

The fraud incident reported at NDB Bank in Sri Lanka highlighted how internal activities can create significant risks for financial institutions. Initial reports estimated the financial impact at approximately Rs. 380 million. Later reports indicated that the impact was much larger and reached approximately Rs. 13.2 billion (NDB Bank, 2026). The incident attracted considerable public attention because it demonstrated that security risks are not always caused by external hackers. Reports suggested that employees acted together with external parties and weaknesses in monitoring procedures allowed suspicious activities to continue for a period of time (Daily FT, 2026). The impact of such incidents extends beyond direct financial losses. Additional consequences may include:

- damage to the reputation of the bank,
- loss of customer trust,
- increased regulatory attention,
- operational disruptions,

- investigation and recovery costs.

The incident also shows that suspicious activities may remain hidden if monitoring systems cannot connect information from multiple sources and identify abnormal behavior patterns.

1.3 Problem Statement

Modern banking systems generate thousands of activities and transactions every day. Monitoring these activities manually becomes extremely difficult because of the volume of information being produced. Many traditional monitoring systems depend on relational databases, predefined rules, and scheduled auditing processes. These approaches perform effectively for routine business operations but may experience limitations when used for real-time fraud detection. Several challenges can be identified:

- Large volumes of transactions create background noise and make suspicious activities difficult to identify.
- Delayed auditing may detect fraud only after financial damage has already occurred.
- Monitoring systems often analyze isolated events instead of connected behavior patterns.
- Introducing new fraud indicators may require database modifications and additional development work.
- Complex database queries may increase processing time when handling large datasets.

As fraud methods continue to change, monitoring systems must also adapt quickly. Systems with rigid structures may become difficult to modify and may reduce the speed of response.

1.4 Objectives

The main objective of this project is to investigate the use of NoSQL technologies for building a real-time insider threat monitoring system for banking environments. The specific objectives are:

- To identify challenges associated with traditional fraud monitoring systems.
- To examine limitations of relational database systems for fraud detection.
- To develop a NoSQL-based monitoring system for handling banking events.
- To implement anomaly detection rules using risk scoring methods.
- To evaluate system performance using simulated banking scenarios.

1.5 Question

This study seeks to answer the following question:

"Can a NoSQL-based real-time monitoring architecture improve the detection and response capabilities for insider banking fraud scenarios?"

1.6 Chapter Summary

This chapter introduced the growing importance of insider threat detection within modern banking systems and presented the NDB fraud incident as a motivating case study. The discussion identified several limitations associated with conventional monitoring approaches, including delayed auditing procedures and challenges in processing large-scale behavioral data. The following chapter investigates the underlying causes of system failures and examines the operational gaps that allow insider threats to remain undetected.

Chapter 2

Root Cause Analysis of System Failure

2.1 Insider Threat Characteristics

Insider threats represent one of the most difficult categories of security incidents to detect within financial institutions because they originate from users who already possess authorized access privileges. Unlike external attackers, insiders typically operate using legitimate credentials, approved devices, and normal operational channels. Consequently, identifying malicious activities from routine employee behavior becomes significantly more challenging.

Insider threats can generally be categorized into three broad groups:

- **Malicious insiders** : employees intentionally abusing access privileges for financial gain or sabotage.
- **Compromised insiders** : legitimate users whose credentials have been stolen or misused by attackers.
- **Negligent insiders** : users unintentionally creating security risks through poor security practices.

Banking environments present a particularly attractive target because employees often possess access to sensitive customer records, transaction systems, internal approval mechanisms, and financial workflows. Since insider activities frequently occur using valid credentials and familiar systems, conventional security mechanisms may incorrectly classify these actions as legitimate operations (Cappelli et al., 2012). Traditional monitoring approaches often examine individual events independently. However, fraudulent behavior rarely occurs as a single isolated action. Instead, insider attacks typically emerge through sequences of seemingly normal activities occurring over time.

For example:

1. Employee login activity at unusual hours
2. Access to unfamiliar customer accounts
3. Creation of new beneficiaries
4. Multiple transfer approvals
5. Rapid transaction execution

Individually, each event may appear normal. When correlated together, these activities may indicate suspicious behavior.

2.2 Operational Monitoring Gaps

Modern banking institutions employ audit systems, transaction logs, and access monitoring procedures intended to identify unusual activities. Despite these controls, several operational weaknesses can reduce detection effectiveness.

2.2.1 Weekend and After Hours Monitoring Limitations

Many fraudulent activities occur during periods of reduced operational oversight such as weekends, public holidays, and late night working hours. During these periods:

- Fewer personnel may be available for review
- Manual verification processes may be delayed
- Escalation procedures may operate at reduced capacity

Attackers often exploit these windows because delayed responses provide additional time to complete fraudulent activities before intervention occurs. The system developed in this study specifically incorporates temporal behavior analysis through after hours detection rules in order to identify activities occurring outside expected operational patterns.

2.2.2 Delayed Audit Procedures

Traditional auditing approaches frequently rely on scheduled reviews and batch processing methods rather than continuous monitoring mechanisms.

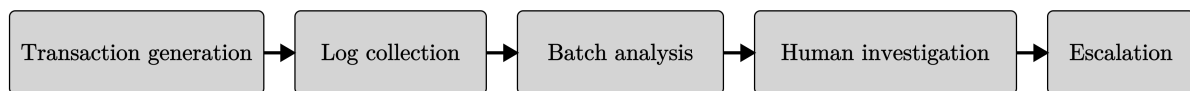


Figure 2.1: Typical operational workflow

While such approaches can identify known issues, they introduce delays between event occurrence and detection. For high value banking environments, even relatively short delays may significantly increase financial exposure.

2.2.3 Lack of Event Correlation

Many existing systems evaluate events independently rather than examining relationships between multiple activities. For example:

- Login activity may be stored separately from transaction records
- Employee information may exist in independent systems
- Customer information may be isolated in separate databases

As a result, identifying coordinated patterns frequently requires combining information across multiple systems. Failure to correlate events may cause systems to overlook complex attacks composed of numerous individually harmless actions.

2.3 High Volume Banking Environment Challenges

Large financial institutions generate substantial quantities of operational data every day. These data include:

- Customer transactions
- Employee actions
- Login records

- System events
- Device information
- Audit logs

Processing high volumes of heterogeneous information introduces significant analytical challenges. A common problem can be described as the *signal versus noise problem*. In environments containing thousands or millions of events, meaningful indicators of fraud represent only a small proportion of overall activity.



Figure 2.2: Signal versus noise problem in banking environments

As transaction volumes increase, identifying suspicious patterns using manual approaches becomes increasingly impractical. Consequently, fraud detection systems require automated methods capable of continuously analyzing behavioral patterns across multiple dimensions.

2.4 Regulatory Considerations and Rapid Response Requirements

Financial institutions operate under strict regulatory requirements intended to minimize operational risks and ensure customer protection.

Regulatory bodies increasingly require institutions to:

- Detect suspicious activities rapidly
- Maintain audit trails
- Report incidents promptly
- Demonstrate appropriate monitoring mechanisms

Rapid detection capabilities are therefore becoming a fundamental operational requirement rather than merely a technical preference.

Delayed identification of suspicious behavior may increase:

- Financial losses
- Recovery costs

- Legal liabilities
- Reputational damage
- Regulatory consequences

These requirements create demand for monitoring systems capable of operating continuously and responding in near real-time.

2.5 Chapter Summary

This chapter examined several operational weaknesses that contribute to failures in traditional fraud detection systems. Insider threats were identified as particularly challenging because they frequently involve legitimate users operating through approved channels. Additionally, delayed audit procedures, lack of event correlation, and increasing transaction volumes create significant obstacles for conventional monitoring systems. The following chapter investigates the limitations of traditional relational database architectures and explains why these environments struggle to support dynamic real-time fraud detection systems.

Chapter 3

Limitations of Traditional SQL Systems

3.1 Schema Rigidity

Relational databases have been widely used in banking systems because they provide structured storage and strong transactional consistency. These characteristics work well when business requirements remain stable. However, fraud detection environments continuously change because attackers regularly introduce new patterns and techniques. For example, a bank may initially store basic transaction information such as employee identifiers, timestamps, transaction amounts, and customer details. Later, security teams may decide that additional information such as device fingerprints, browser details, cryptocurrency wallet addresses, or geolocation information should also be tracked. In SQL systems, introducing such changes commonly requires modifying the database schema.

Listing 3.1: Adding new attributes in SQL

```
1 ALTER TABLE employee_actions
2 ADD COLUMN device_fingerprint VARCHAR(64);
3
4 ALTER TABLE beneficiaries
5 ADD COLUMN crypto_wallet VARCHAR(100);
```

Although schema modifications appear simple, changes become more difficult when databases contain millions of records. Database administrators may need to perform migration procedures, execute testing activities, and schedule maintenance windows before deployment (Silberschatz et al., 2019). For banking environments this creates operational challenges because many systems run continuously and cannot easily tolerate downtime. Even short interruptions may affect customer transactions and service availability. The financial impact can also be significant. Delays in transaction processing may lead to customer dissatisfaction, increased support costs, and operational disruption. Since fraud patterns evolve continuously, a system that requires structural changes whenever new indicators appear may respond slowly to emerging threats.

3.2 Join Complexity and Query Performance

Fraud detection rarely depends on a single event. Suspicious behavior often becomes visible only after combining multiple activities together. For example, a monitoring system may need to identify:

- Which employee performed an action,
- Whether multiple employees used the same IP address
- Whether a customer account was recently created
- Whether unusual transfers occurred

In relational systems, this information is frequently distributed across multiple tables. A simplified fraud detection query is shown below.

Listing 3.2: Example SQL fraud analysis query

```

1 SELECT
2     a1.employee_id,
3     a2.employee_id,
4     a1.ip_address,
5     t.amount,
6     b.name
7 FROM employee_actions a1
8
9 INNER JOIN employee_actions a2
10 ON a1.ip_address = a2.ip_address
11
12 LEFT JOIN transactions t
13 ON a1.action_id = t.action_id
14
15 LEFT JOIN beneficiaries b
16 ON t.beneficiary_id = b.beneficiary_id
17
18 WHERE a1.action_type='LOGIN';

```

The query itself is not necessarily complex, but difficulties appear as datasets increase in size. Banking systems may process thousands or millions of records daily. As the amount of information grows, multiple JOIN operations require additional processing resources (Elmasri and Navathe, 2017). In fraud detection environments, response time becomes important because suspicious activities should ideally be identified while transactions are still occurring rather than after completion. A delay of several seconds may appear small from a technical perspective, but in banking systems this delay can have wider consequences including:

- delayed fraud response,
- interrupted customer transactions,
- increased investigation effort,
- additional financial losses.

3.3 Batch Processing Limitations

Many traditional monitoring systems rely on batch processing approaches rather than continuous real-time analysis. In a batch-based environment, events are collected over a period of time and analyzed later. Instead of examining activities immediately, the system processes accumulated records at scheduled intervals.

The general process often follows the sequence shown below:



Figure 3.1: General process sequence

This approach may work adequately for reporting and auditing purposes, but it creates delays in security monitoring. Fraud activities can occur within a short period of time. If analysis takes place several hours later, attackers may already have completed transactions and moved funds before the system reacts. Since modern banking environments generate large volumes of events continuously, relying only on delayed processing creates challenges for rapid response and real-time monitoring.

3.4 Development Complexity

Modern applications are commonly developed using object-oriented programming languages such as Python, Java and C#. These programming languages represent information using objects, while relational databases store information as rows and tables. Because of this difference, developers often use Object Relational Mapping (ORM) frameworks to convert application objects into database records. An example is shown below.

Listing 3.3: Typical SQL data conversion process (python)

```
1 results = []
2
3 for row in cursor.fetchall():
4
5     action = {
6         "employee_id": row[0],
7         "timestamp": row[1],
8         "transaction": {
9             "amount": row[2]
10        }
11    }
12
13    results.append(action)
```

Although ORM tools simplify development, they introduce additional mapping layers between the application and database. This issue is commonly referred to as the object-relational mismatch problem (Ambler, 2003). As systems become larger, developers frequently need to maintain additional configuration logic, relationships, and data conversions. In systems where fraud rules change frequently, this additional effort may slow development and increase maintenance requirements.

3.5 Chapter Summary

This chapter examined several limitations associated with traditional SQL systems in fraud detection environments. Fixed schemas reduce flexibility when new fraud indicators need to be introduced, while JOIN operations may increase processing time as datasets grow larger. Additionally, batch processing approaches can introduce delays in detecting suspicious activities, and object-relational differences may increase development complexity. These limitations create challenges for building systems that require flexible data structures and near real-time analysis. The following chapter discusses how NoSQL databases address many of these issues and why they provide an alternative approach for modern fraud detection systems.

Chapter 4

NoSQL as a Proposed Solution

4.1 Characteristics of NoSQL Databases

NoSQL databases were introduced to address several limitations found in traditional relational systems when handling large and rapidly changing datasets. Unlike SQL databases, NoSQL systems do not depend heavily on fixed table structures and predefined relationships. Instead, they provide greater flexibility when storing and retrieving information. One important characteristic of NoSQL databases is schema flexibility. Traditional relational systems usually require a predefined structure before data can be stored. However, fraud detection environments continuously change because attackers regularly introduce new techniques and behavior patterns. For example, a system may initially track only transaction amounts and customer information. Later, security teams may decide that additional information such as IP addresses, device identifiers, browser information, or geolocation data should also be monitored. In a relational system this may require database migrations and schema modifications. In contrast, NoSQL systems allow new information to be added directly into documents without affecting previously stored data. A simple example is shown below.

Listing 4.1: Adding new information to a document

```
1 {  
2   "employeeId": "EMP102",  
3   "actionType": "TRANSFER",  
4   "timestamp": "2026-04-06T23:15:00",  
5  
6   "location": {  
7     "ipAddress": "192.168.1.45"  
8   },  
9  
10  "deviceFingerprint": "abc123xyz"  
11 }
```

Older records remain unchanged while newer records can include additional attributes. This flexibility allows systems to adapt more quickly when new fraud indicators are identified. Another important characteristic is horizontal scalability. Instead of increasing resources on a single server, NoSQL databases can distribute data across multiple systems. As transaction volume increases, additional machines can be introduced to handle larger workloads (Han et al., 2011). NoSQL databases also commonly use document-oriented architectures where related information can be stored together inside a single structure. This approach becomes useful in fraud monitoring environments because security events frequently contain different types of information and do not always follow a fixed format.

4.2 Why MongoDB Was Selected

Several NoSQL technologies currently exist, including key-value databases, graph databases, and document databases. For this implementation, MongoDB was selected because its characteristics closely match the requirements of real-time fraud monitoring. MongoDB stores information as BSON documents which are similar to JSON structures. Since the proposed system was

developed using Python, this creates a natural flow between application objects and database documents. An employee activity record in the implemented system can be represented as:

Listing 4.2: Employee activity document

```
1 {
2   "employeeId": "EMP102",
3   "actionType": "TRANSFER",
4
5   "timestamp": "2026-04-06T23:15:00",
6
7   "location": {
8     "ipAddress": "192.168.1.45"
9   },
10
11  "relatedTransaction": {
12    "customerId": "CUS450",
13    "amount": 5000000,
14    "beneficiary": "Buy Today"
15  }
16 }
```

Unlike relational systems where this information may be separated across several tables, MongoDB allows related details to remain within a single document. This structure reduces the need for multiple JOIN operations and simplifies information retrieval. MongoDB was also selected because of its indexing capability. Fraud detection systems repeatedly search for patterns involving timestamps, employee identifiers, and login locations. Searching without indexes may require scanning large amounts of information. An example index used for employee monitoring is shown below.

Listing 4.3: MongoDB compound index

```
1 db.employee_actions.createIndex(
2 {
3   "employeeId": 1,
4   "timestamp": -1
5 }
6 )
```

This index allows the system to quickly retrieve activities related to a specific employee while maintaining efficient time-based searches. Since the monitoring dashboard continuously requests recent activities, efficient indexing becomes important for maintaining fast response times.

4.3 Benefits for Fraud Detection Systems

Fraud detection systems require continuous monitoring rather than delayed analysis after transactions have already completed. One major advantage of MongoDB is the ability to process event-driven information efficiently. In the developed system, employee activities are continuously generated and inserted into MongoDB while the detection engine evaluates each event immediately. Instead of waiting for scheduled auditing procedures, the system can react when activities occur. For example, the system may immediately search for employee activities originating from the same IP address within a recent time period.

Listing 4.4: Searching suspicious employee activity

```
1 db.employee_actions.find({
2
3   "location.ipAddress": "192.168.1.45",
```

```
4
5 "timestamp":{
6 "$gte":ISODate("2026-04-06T23:10:00")
7 }
8
9 })
```

The query directly retrieves suspicious activity without requiring multiple JOIN operations. Another advantage is that detection rules can expand as new attack patterns emerge. The implemented system currently includes several behavioral checks such as after-hours activity detection, suspicious beneficiary monitoring, credential misuse detection, and ghost account identification. If new fraud patterns appear in the future, additional fields and rules can be introduced without redesigning the database structure. This ability to evolve quickly becomes important because fraud techniques continuously change over time. Rather than forcing developers to redesign databases whenever requirements change, MongoDB provides a flexible environment that supports continuous adaptation.

4.4 Chapter Summary

This chapter introduced NoSQL databases as an alternative approach for overcoming several limitations observed in traditional relational systems. Features such as flexible schemas, horizontal scalability, and document-oriented storage provide advantages for environments where data structures frequently change. MongoDB was selected for this implementation because of its JSON-like document structure, efficient querying capability, and compatibility with real-time applications. These characteristics support the requirements of fraud monitoring systems that must process large volumes of events while adapting rapidly to new threat patterns. The following chapter explains the design and implementation of the proposed fraud detection system developed for this study.

Chapter 5

System Design and Implementation

Source Code Availability: The complete source code, database generation scripts, and execution instructions for this NoSQL monitoring engine and SOC terminal are publicly available on GitHub at: <https://github.com/Niroshan-k/Insider-Threat-Detection-for-Banks.git>

5.1 Overall System Architecture

5.1.1 Architectural Overview

This chapter explains the design and implementation of the proposed insider threat detection system. The system was developed to simulate a banking environment and evaluate how real-time monitoring can identify suspicious employee activities associated with insider fraud. The implementation follows a modular architecture where each component performs a specific task. Instead of combining all functionality into a single program, the system separates data generation, rule evaluation, database management, and monitoring activities into independent modules. This approach improves maintainability and allows individual components to be modified without affecting the entire system. The proposed architecture consists of three major components: World Engine, Rule Engine, Monitoring Dashboard.

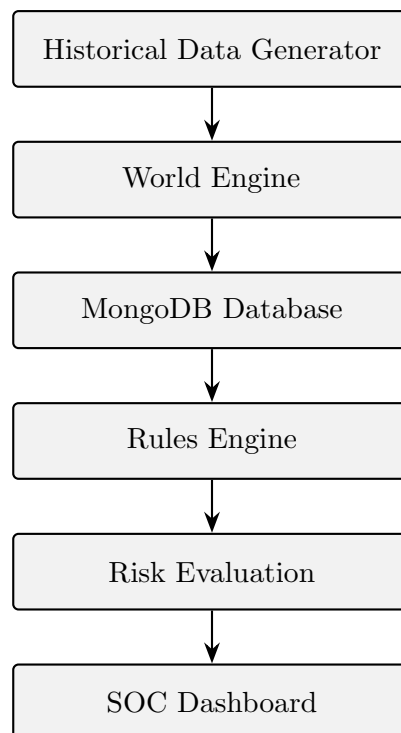


Figure 5.1: Overall architecture of the proposed threat detection system.

The World Engine is responsible for creating the simulated banking environment. It generates employee profiles, customer records, transaction activities, and historical behavioral information required for the monitoring process. The Rules Engine continuously evaluates employee actions and calculates risk scores using predefined detection rules. The rules were designed based on patterns observed in insider threat scenarios such as after-hours access, credential misuse, and suspicious transaction behavior. The Monitoring Dashboard acts as the presentation layer of the system. Information retrieved from the database is processed and displayed through a real-time terminal interface which provides visual feedback regarding risk levels and alert conditions. Separating these components creates a clear processing flow where generated activities are stored within MongoDB, analyzed by the detection engine, and finally displayed to security analysts through the monitoring interface.

5.1.2 World Engine

The World Engine was implemented to create a realistic simulation environment representing normal banking activities. Since obtaining access to real banking datasets is difficult because of privacy and security restrictions, synthetic data generation techniques were used. The implementation automatically creates employee records, customer profiles, and historical transaction activities before the monitoring process begins. The following code segment from the implementation demonstrates employee generation.

Listing 5.1: Employee profile generation

```
1 employees = []
2
3 for i in range(1,99):
4
5     employees.append({
6
7         "employeeId":f"EMP_{1000+i}",
8
9         "department": random.choice(departments),
10        "role": random.choice(roles),
11        "baseline":{
12
13            "typicalWorkHours":{
14                "start":8, "end":17
15            },
16
17            "avgApprovalsPerDay": random.randint(1,10)
18        }
19    })
```

The generated employee records include behavioral baseline information such as working hours and average daily approvals. These values are later used by the detection engine when evaluating suspicious activities. Two predefined employee records were also introduced manually within the simulation environment.

Listing 5.2: Injected employee records

```
1 saman_id = "EMP_4821"
2
3 kamal_id = "EMP_7721"
```

These employees were used during fraud simulations to recreate insider attack scenarios involving credential compromise and unauthorized activities.

5.1.3 Rules Engine

The Rules Engine is responsible for analyzing employee activities and assigning risk values according to predefined security rules. The implementation continuously receives employee activities from MongoDB and evaluates them against several behavioral conditions. The main processing logic is implemented using the following function:

Listing 5.3: Rules engine entry function

```
1 def evaluate_employee_action( action, db):  
2     risk_score = random.uniform( 0.05, 0.15 )  
3  
4     flags = []  
5  
6     ...
```

The engine initially creates a baseline risk value and gradually increases the score whenever suspicious behavior is detected. Different detection rules contribute additional penalties depending on the activity being observed. The calculated score and generated alert information are returned to the monitoring system where they are displayed to security analysts. This approach allows multiple suspicious indicators to be combined rather than relying on a single event.

5.1.4 Monitoring Dashboard

The Monitoring Dashboard acts as the presentation layer of the system. Its purpose is to provide real-time visibility into employee activities and generated security alerts. The dashboard was implemented using Streamlit and continuously retrieves recent events from MongoDB. The implementation retrieves recent activities as shown below.

Listing 5.4: Retrieving recent employee actions

```
1 recent_actions = list( db.employee_actions.find().sort( "timestamp",  
    -1).limit(150))
```

The retrieved records are passed into the Rules Engine where risk values are calculated before being displayed. The dashboard presents information including transaction activity, risk scores, event distribution, and critical alerts. By continuously updating the displayed information, security personnel can monitor system activity and quickly identify unusual behavior patterns.

5.2 Dataset and Simulation Environment

Obtaining access to real banking datasets is difficult because financial information contains sensitive customer and employee data. Due to privacy restrictions and security concerns, publicly available datasets rarely contain detailed insider threat activities. Therefore, a synthetic simulation environment was developed for this study. The purpose of the simulation environment was to create realistic banking activities while maintaining control over employee behavior and fraud scenarios. The generated environment contains employee records, customer profiles, historical transaction data, and manually injected attack scenarios. The generated dataset was designed to resemble normal banking activities while also allowing suspicious patterns to be introduced for evaluation purposes.

5.2.1 Employee Profile Generation

The first stage of the simulation process creates employee profiles representing bank staff members from different departments. The implementation randomly generates employees and assigns information such as department, role, and baseline behavior characteristics.

Listing 5.5: Employee profile generation

```

1 departments = [
2     "TREASURY",
3     "RETAIL_BANKING",
4     "IT_OPS",
5     "PAYMENTS_AND_SETTLEMENTS",
6     "HR"
7 ]
8
9 roles = [
10    "OFFICER",
11    "SENIOR_OFFICER",
12    "MANAGER",
13    "ASST_MANAGER"
14 ]
15
16 for i in range(1,20):
17     employees.append({
18         "employeeId": f"EMP_{1000+i}",
19
20         "department": random.choice(departments),
21
22         "role": random.choice(roles),
23
24         "baseline":{
25             "typicalWorkHours":{
26                 "start":8,
27                 "end":17
28             },
29
30             "avgApprovalsPerDay":
31                 random.randint(1,10)
32         }
33     })

```

Listing 5.6: Mongosh Insert operation example

```

1 db.employees.insertOne({
2
3     employeeId: "EMP_1001",
4     department: "TREASURY",
5     role: "OFFICER",
6
7     baseline:{
8         typicalWorkHours:{
9             start:8,
10            end:17
11        },
12        avgApprovalsPerDay:5
13    }
14 })

```

Each employee contains a baseline profile that describes expected working behavior. Typical working hours and average daily approval values are included because these values are later used by the detection engine when evaluating suspicious activities. Two predefined employee identities were additionally inserted into the generated dataset for fraud simulation purposes.

Listing 5.7: Injected employee identities

```

1 saman_id = "EMP_4821"
2
3 kamal_id = "EMP_7721"

```

These employee records were introduced to recreate insider attack scenarios involving credential misuse and unauthorized activities.

5.2.2 Customer Dataset Generation

After generating employees, the system creates customer records that simulate regular banking users. The implementation generates 500 customer profiles containing account information and Know Your Customer (KYC) verification status.

Listing 5.8: Customer profile generation

```

1 for i in range(500):
2
3     customers.append({
4
5         "customerId": f"CUST_{10000+i}",
6         "name": fake.company(),
7
8         "createdAt": start_of_week - timedelta
9         (
10             days=random.randint( 30, 3000 )
11         ),
12
13         "kycVerification": True
14     })

```

Listing 5.9: Mongosh retrieval example for not verified

```

1 db.customers.find(
2     { kycVerification: false }
3 )

```

Most generated customers represent legitimate users with completed verification information. However, one customer account was intentionally created to simulate suspicious behavior.

Listing 5.10: Ghost account injection

```

1 customers.append({
2
3     "customerId": "CUST_GHOST_01",
4     "name": "Buy Today Holdings",
5     "createdAt": start_of_week + timedelta(days=4),
6     "kycVerification": False
7
8 })

```

This account represents a recently created customer profile without completed KYC verification. The account is later used during fraud simulations to evaluate whether the detection engine correctly identifies abnormal activities.

5.2.3 Historical Transaction Generation

After generating employee and customer records, the system creates historical banking activities. The implementation generates seven days of normal transaction behavior before introducing suspicious activities. Generated events include customer queries, account views, employee logins, loan approvals, and electronic transfers.

Listing 5.11: Historical activity generation

```

1 action_type = random.choice([
2
3     "CUSTOMER_QUERY",
4     "ACCOUNT_VIEW",
5     "LOGIN",
6     "LOAN_APPROVAL",
7     "ELECTRONIC_TRANSFER"
8
9 ])

```

Listing 5.12: Mongosh query example

```

1 db.employee_actions.find(
2 {
3     actionType:"LOGIN"
4 })
5 .sort(
6 {
7     timestamp:-1
8 })
9 .limit(5)

```

The generated activities are intentionally constrained to normal business hours.

Listing 5.13: Business hour generation

```

1 random_day = start_of_week + timedelta
2 (
3     days=random.randint(0,4)
4 )
5
6 random_time = random_day + timedelta
7 (
8     hours=random.randint(8,16),
9     minutes=random.randint(0,59)
10 )

```

Restricting activities to business periods creates stable behavioral patterns that can later be used by the rules engine. Since most historical activities represent normal behavior, suspicious activities become easier to identify when anomalies occur.

5.2.4 Fraud Scenario Injection

Normal transaction activities alone are not sufficient for evaluating fraud detection systems because suspicious activities may not naturally appear within generated data. To address this limitation, predefined fraud scenarios were introduced into the simulation environment. These scenarios were designed based on insider threat patterns associated with the NDB-style fraud scenario discussed earlier in this study. The injected scenarios include credential misuse, high-value transfers, unverified customer accounts, and unusual employee activities. These activities were intentionally introduced to reproduce behaviors commonly associated with insider threat incidents. The inclusion of manually injected attacks allows the proposed system to evaluate whether the detection engine correctly identifies abnormal patterns and increases risk scores accordingly. The generated dataset forms the foundation of the proposed monitoring system because the detection engine depends on these activities when calculating risk scores and identifying insider threats.

5.3 Detection Rule Engine

The Detection Rule Engine is responsible for evaluating employee activities and determining whether a particular action represents suspicious behavior. Instead of depending on a single indicator, the implemented system uses multiple rules that examine different characteristics of employee actions. Each rule contributes additional penalties to a risk score whenever abnormal behavior is detected. If the final score exceeds a predefined threshold, the event is marked as suspicious and displayed through the monitoring dashboard. The implementation begins by assigning a small baseline risk score to every incoming activity.

Listing 5.14: Initial risk score generation

```
1 risk_score = random.uniform( 0.05, 0.15 )
2
3 flags = []
```

As different rules detect suspicious conditions, additional values are added to the score while descriptive flags are stored for later analysis.

5.3.1 After-Hours Activity Detection

One common characteristic of insider threats is unusual activity outside normal working periods. Employees generally follow predictable schedules and significant deviations from these patterns may indicate suspicious behavior. The developed system evaluates the timestamp associated with each employee activity and determines whether the event occurs outside expected working hours.

Listing 5.15: After-hours activity detection

```
1 hour = action["timestamp"].hour
2
3 if hour < 8 or hour > 17:
4
5     risk_score += 0.30
6
7     flags.append(
8
9         "After Hours Activity"
10    )
```

If the employee performs activities during late-night periods or outside predefined business hours, the risk score increases and a corresponding warning flag is generated. This rule helps identify situations where compromised accounts or unauthorized access attempts occur during periods of reduced monitoring.

5.3.2 Credential Abuse Detection

Credential theft and account misuse represent common insider threat techniques. In some situations, attackers obtain employee login information and perform actions using stolen credentials. To identify this behavior, the system examines employee activities originating from the same IP address within a short time period.

Listing 5.16: Credential abuse detection

```
1 similar_logins = db.employee_actions.find({
2
3     "location.ipAddress": action["location"]["ipAddress"],
4
```

```

5     "timestamp":{
6         "$gte": action["timestamp"] - timedelta(minutes=5)
7     }
8
9 })

```

Listing 5.17: Mongosh detection query example

```

1 db.employee_actions.find({
2
3     "location.ipAddress":"10.0.0.99",
4
5     timestamp:{
6         $gte: ISODate( "2026-04-06T23:00:00Z" )
7     }
8 })

```

The system retrieves recent activities and checks whether multiple employee identities appear from the same source.

Listing 5.18: Credential abuse identification

```

1 employee_ids = set( a["employeeId"] for a in similar_logins )
2
3 if len(employee_ids) > 1:
4
5     risk_score += 0.60
6     flags.append(
7
8         "Credential Abuse"
9     )

```

Multiple employee identities appearing from the same IP address within a short time interval may indicate suspicious account usage.

5.3.3 Suspicious Beneficiary Detection

Fraud activities frequently involve transferring funds to unfamiliar beneficiaries or external entities that have not previously interacted with the banking system. The implemented system examines beneficiary information associated with transactions and determines whether the entity appears unusual.

Listing 5.19: Suspicious beneficiary detection

```

1 beneficiary = action.get( "relatedTransaction", {} ).get( "
    beneficiaryName", "" )
2
3 if "crypto" in beneficiary.lower():
4
5     risk_score += 0.40
6
7     flags.append(
8
9         "Suspicious Beneficiary"
10    )

```

Transactions involving cryptocurrency-related entities or unfamiliar destinations receive additional penalties because such activities may represent attempts to move funds rapidly.

5.3.4 Ghost Account Detection

Fraud schemes often involve recently created accounts with incomplete verification information. These accounts may be used to temporarily receive funds before transactions are redirected elsewhere. The implemented system checks customer verification information stored in the database.

Listing 5.20: Ghost account detection

```
1 customer = db.customers.findOne({ "customerId": action["  
    relatedTransaction"]["customerId"] })  
2  
3 if customer and not customer["kycVerification"]:  
4  
5     risk_score += 0.50  
6  
7     flags.append(  
8         "Ghost Account"  
9     )
```

Listing 5.21: Mongosh lookup example

```
1 db.customers.findOne({  
2     customerId:"CUST_GHOST_01"  
3 })
```

Accounts that do not satisfy KYC requirements increase the overall risk score and generate an additional warning indicator. This rule was introduced specifically to reproduce conditions associated with the simulated NDB-style fraud scenario.

5.3.5 Composite Risk Score Model

The proposed system does not rely on individual rules independently because isolated events may appear normal in many situations. Instead, the final risk value is calculated by combining penalties from multiple rules. The general scoring process used by the implementation is shown below.

Listing 5.22: Composite risk calculation

```
1 final_score = ( base_score + after_hours + credential_abuse +  
    beneficiary_risk + ghost_account )
```

The final score is compared against predefined thresholds.

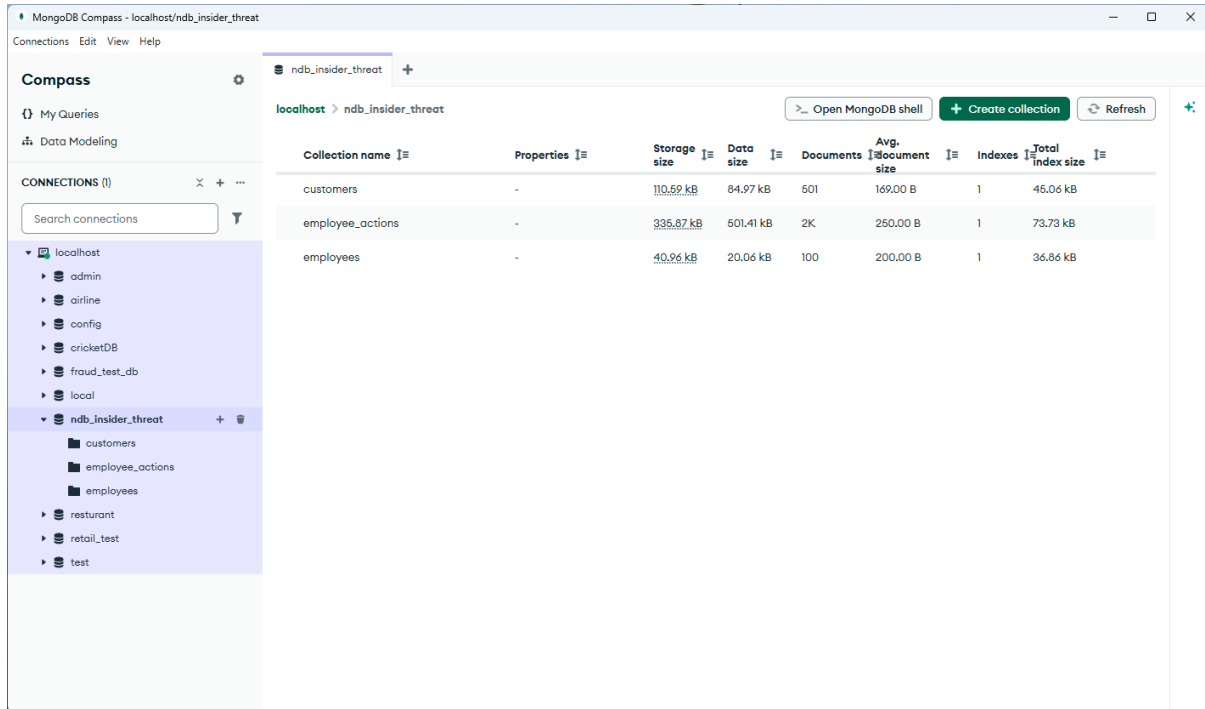
Listing 5.23: Alert threshold evaluation

```
1 if risk_score >= 0.75:  
2  
3     alert = True  
4  
5 else:  
6  
7     alert = False
```

Activities exceeding the threshold are classified as suspicious and forwarded to the monitoring dashboard. Using a composite scoring approach allows the system to combine several weak indicators into a stronger decision process. Rather than generating alerts from a single event, the proposed approach evaluates multiple behavioral characteristics simultaneously and reduces the possibility of false detections.

5.4 MongoDB Schema Design

The proposed system stores information using MongoDB collections rather than relational tables. Since the application continuously generates employee activities and evaluates behavior patterns, a document-oriented database structure provides flexibility and simplifies data retrieval. The implementation uses three primary collections: `employee_actions`, `employees`, and `customers`.



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists various databases, with 'ndb_insider_threat' selected. The main panel displays the 'ndb_insider_threat' database structure. A table lists the collections: 'customers', 'employee_actions', and 'employees'. Each row provides details on storage size, data size, document count, average document size, index count, and total index size.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
customers	-	110.59 kB	84.97 kB	501	169.00 B	1	45.06 kB
employee_actions	-	335.87 kB	501.41 kB	2K	250.00 B	1	73.73 kB
employees	-	40.96 kB	20.06 kB	100	200.00 B	1	36.86 kB

Figure 5.2: MongoDB Compass

5.4.1 employee_actions Collection

The `employee_actions` collection acts as the main event log within the system. Every employee activity generated by the simulation environment is inserted into this collection. Each document stores information related to employee behavior and transaction details.

Listing 5.24: employee_actions document structure

```
1 {  
2  
3   "employeeId": "EMP_4821",  
4   "actionType": "LOGIN",  
5   "timestamp": "2026-04-06T23:15:00",  
6   "location": { "ipAddress": "10.0.0.99" },  
7   "relatedTransaction": {  
8  
9     "customerId": "CUST_GHOST_01",  
10    "amount": 15000000,  
11    "beneficiaryName": "Buy Today Crypto"  
12  }  
13 }  
14 }
```

Embedding related information inside a single document reduces the need for complex JOIN operations and improves retrieval speed during analysis.

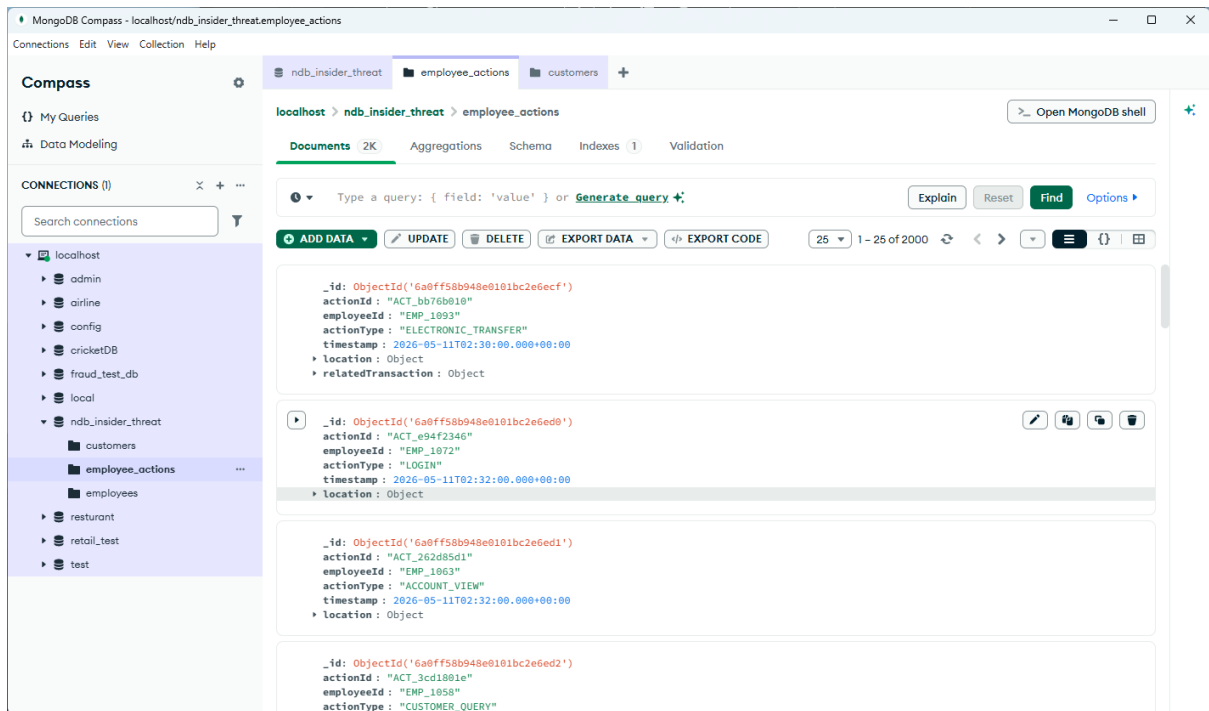


Figure 5.3: MongoDB Compass employee action collection

5.4.2 employees Collection

The `employees` collection stores employee profile information and behavioral baseline values. These values represent normal employee behavior and are later used during risk evaluation.

Listing 5.25: employees document structure

```

1 {
2
3   "employeeId": "EMP_1001",
4   "department": "TREASURY",
5   "role": "OFFICER",
6   "baseline": {
7
8     "typicalWorkHours": {
9       "start": 8,
10      "end": 17
11    },
12
13    "avgApprovalsPerDay": 5
14
15  }
16 }
```

Maintaining baseline information separately allows the system to compare current activities against expected employee behavior.

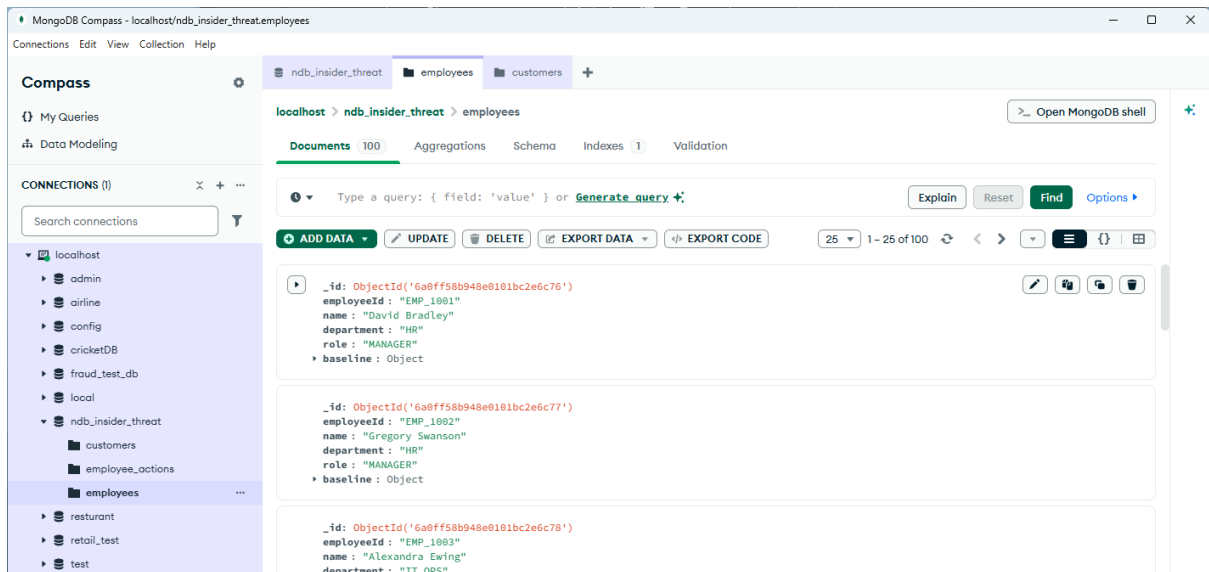


Figure 5.4: MongoDB Compass employee collection

5.4.3 customers Collection

The **customers** collection stores customer profile information together with account verification details. The system uses this information when evaluating suspicious account activities.

Listing 5.26: customers document structure

```

1 {
2   "customerId": "CUST_GHOST_01",
3   "name": "Buy Today Holdings",
4   "createdAt": "2026-04-05",
5   "kycVerification": false
6 }

```

The customer collection plays an important role during ghost account detection because incomplete KYC information increases risk values.

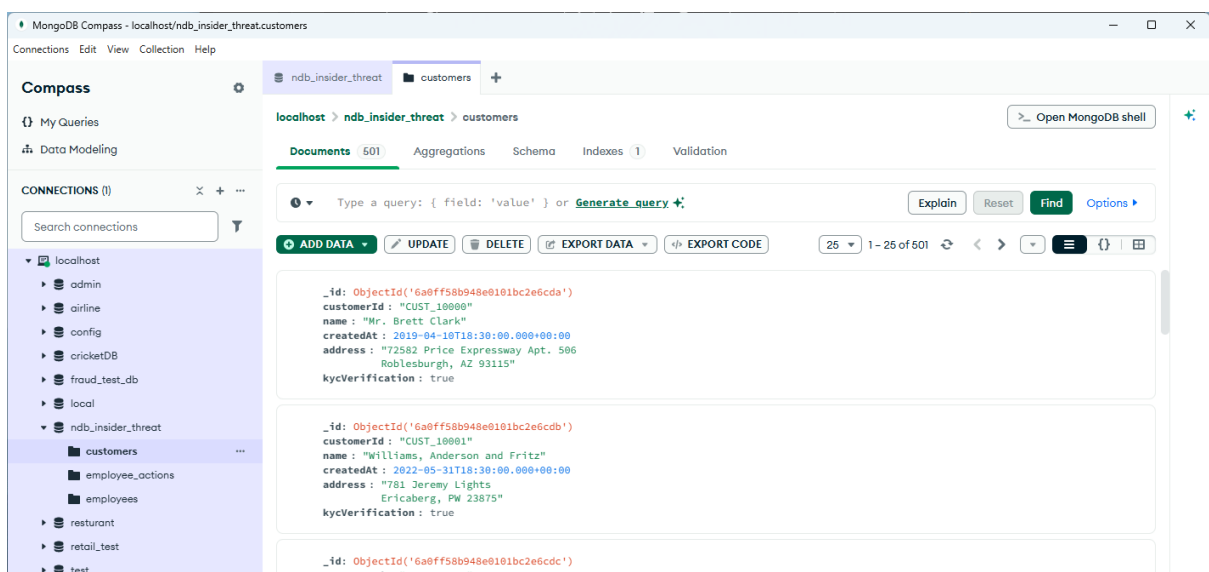


Figure 5.5: MongoDB Compass customer collection

5.4.4 Indexing Strategy

Indexes were introduced to improve retrieval performance for frequently accessed attributes. Since the detection engine repeatedly searches employee identifiers, timestamps, and IP addresses, indexing reduces database scanning operations. The following implementation illustrates the indexing strategy used within the system.

Listing 5.27: MongoDB indexing strategy

```
1 db.employee_actions.create_index(  
2     [("employeeId",1)]  
3 )  
4  
5 db.employee_actions.create_index(  
6     [("timestamp",-1)]  
7 )  
8  
9 db.employee_actions.create_index(  
10    [("location.ipAddress",1)]  
11 )
```

Listing 5.28: Equivalent Mongosh example

```
1 db.employee_actions.createIndex(  
2 {  
3     employeeId:1  
4 })  
5  
6 db.employee_actions.createIndex(  
7 {  
8     timestamp:-1  
9 })  
10  
11 db.employee_actions.createIndex(  
12 {  
13     "location.ipAddress":1  
14 })
```

Listing 5.29: Mongosh delete example

```
1 db.employee_actions.deleteOne({  
2  
3     employeeId:"EMP_4821",  
4     actionType:"LOGIN"  
5  
6 })
```

Indexing allows the system to locate relevant information more efficiently and improves response times during real-time monitoring activities.

5.5 Real-Time Traffic Generator

The proposed system continuously generates employee activities to simulate live banking operations. The purpose of the traffic generator is to create a realistic stream of events rather than relying entirely on static historical information. The implementation periodically creates new activities and inserts them into MongoDB.

Listing 5.30: Live activity generation

```

1 while True:
2
3     action = generate_action()
4     db.employee_actions.insert_one( action )
5
6     time.sleep( random.randint( 1, 3 )
7 )

```

Random intervals between generated activities create a more realistic workload and prevent events from appearing at fixed patterns. This approach produces continuous traffic that allows the monitoring dashboard and detection engine to operate under near real-time conditions.

5.6 SOC Dashboard Design

The Security Operations Center (SOC) dashboard was implemented to provide real-time visibility into employee activities and generated alerts.



Figure 5.6: SOC Dashboard

The dashboard was developed using Streamlit and designed to resemble a Bloomberg-style monitoring terminal. Information is continuously retrieved from MongoDB and displayed through multiple visual components. Recent employee activities are retrieved using the following implementation.

Listing 5.31: Retrieving recent activities

```

1 recent_actions = list( db.employee_actions.find().sort( "timestamp",
2                                     -1).limit(150))

```

The dashboard displays information such as transaction activity, risk scores, event distribution, and critical alerts. Visual indicators use color coding where green represents normal activities while red highlights suspicious behavior requiring attention. This interface allows security personnel to quickly identify abnormal patterns and monitor system behavior continuously.

5.7 Attack Simulation Module

The Attack Simulation Module was developed to reproduce insider threat scenarios and evaluate whether the detection engine correctly identifies suspicious activities.

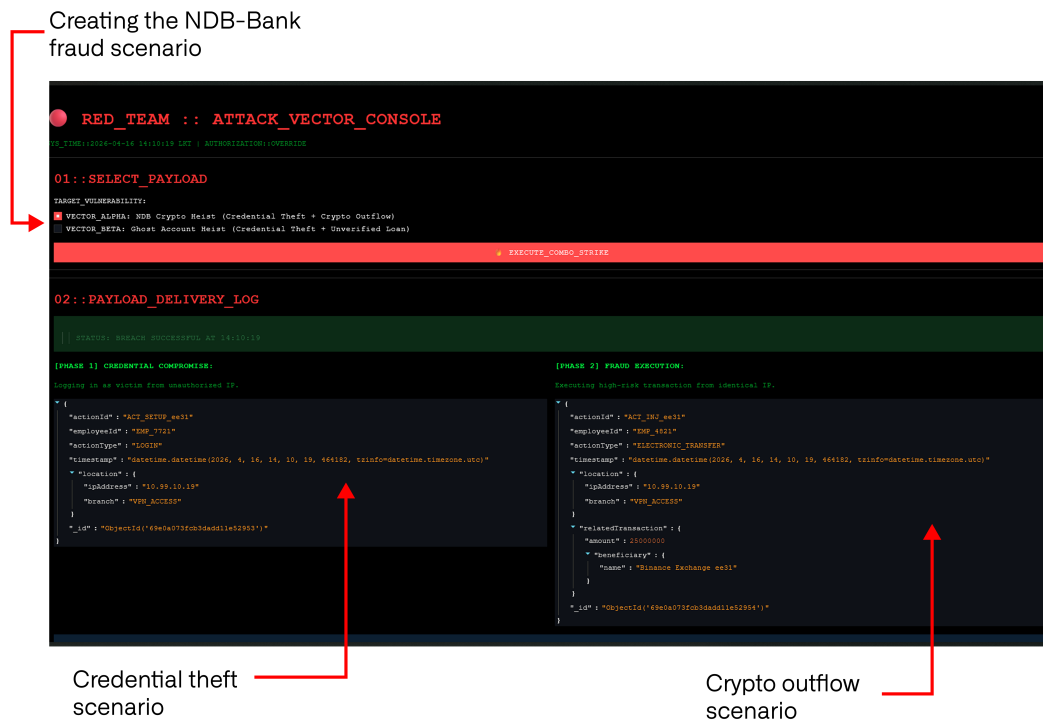


Figure 5.7: Attacker simulation Dashboard

Instead of waiting for naturally occurring fraud events, predefined attack patterns were introduced into the simulation environment. The implementation inserts suspicious employee actions directly into MongoDB.

Listing 5.32: Fraud scenario injection

```

1 attack_event = {
2
3     "employeeId": "EMP_4821",
4     "actionType": "ELECTRONIC_TRANSFER",
5     "timestamp": datetime.now(),
6
7     "relatedTransaction":{
8
9         "customerId": "CUST_GHOST_01",
10        "amount": 15000000
11    }
12 }
13 }
```

```
14 db.employee_actions.insert_one( attack_event )
15
```

Listing 5.33: Mongosh update example

```
1 db.employee_actions.updateOne(
2
3 { employeeId:"EMP_4821" },
4
5 {
6     $set:{
7         actionType:"FLAGGED_TRANSFER"
8     }
9 }
10 )
```

These injected activities reproduce behaviors associated with credential compromise, unauthorized transactions, and ghost account usage. The attack simulation module allows controlled testing of the proposed system and provides a repeatable mechanism for evaluating detection performance.

5.8 Chapter Summary

This chapter presented the design and implementation of the proposed insider threat detection system. The overall architecture was divided into independent components consisting of the World Engine, Rules Engine, and Monitoring Dashboard to create a structured processing workflow. The chapter explained how the simulation environment was developed using synthetic employee profiles, customer records, historical activities, and manually injected fraud scenarios. The Detection Rule Engine was introduced as the main analytical component responsible for evaluating employee behavior through multiple security rules and calculating risk scores. The MongoDB database design was also discussed, including the use of collections and indexing strategies that support efficient retrieval of information. In addition, the implementation of the real-time traffic generator, SOC dashboard, and attack simulation module demonstrated how the system continuously generates, analyzes, and visualizes suspicious activities. Overall, the implemented system provides an integrated environment capable of simulating banking activities and identifying abnormal behavior in near real-time. The next chapter evaluates the proposed system and presents experimental results related to detection performance and system behavior.

Chapter 6

System Evaluation

6.1 Experimental Setup

The purpose of this chapter is to evaluate the behavior and performance of the proposed insider threat detection system. The experiments were conducted using the developed implementation described in Chapter 5. The evaluation focuses on four main areas:

- detection performance,
- system latency,
- scalability,
- overall system behavior.

The implementation was executed using Python, MongoDB, and Streamlit within a local environment. Table 6.1 presents the software environment used during experimentation.

Table 6.1: Experimental environment

Component	Configuration
Programming Language	Python 3.12
Database	MongoDB 7.0
Dashboard Framework	Streamlit 1.30
Data Visualization	Plotly
Operating System	Windows 11

The dataset used during evaluation consisted of:

- 500 customer profiles,
- employee profiles,
- seven days of generated historical activity,
- injected fraud scenarios.

The fraud scenarios included credential misuse, suspicious transfers, and ghost account activities. To obtain performance measurements, experiments were repeated multiple times under identical conditions. Response times were measured using Python timing function `time.perf_counter()` before and after execution of selected operations. Each experiment was executed 100 times and the average values were recorded to reduce the effect of random variation and background system activity.

6.2 Detection Performance

Detection performance was evaluated by observing whether the rules engine correctly identified suspicious activities inserted into the system. The system generated risk scores based on predefined rules and classified activities as normal or suspicious according to the configured threshold values.

Table 6.2 presents observed detection behavior.

Table 6.2: Detection results

Scenario	Detection Status	Alert Generated
After-hours activity	Detected	Yes
Credential abuse	Detected	Yes
Ghost account usage	Detected	Yes
Suspicious beneficiary	Detected	Yes

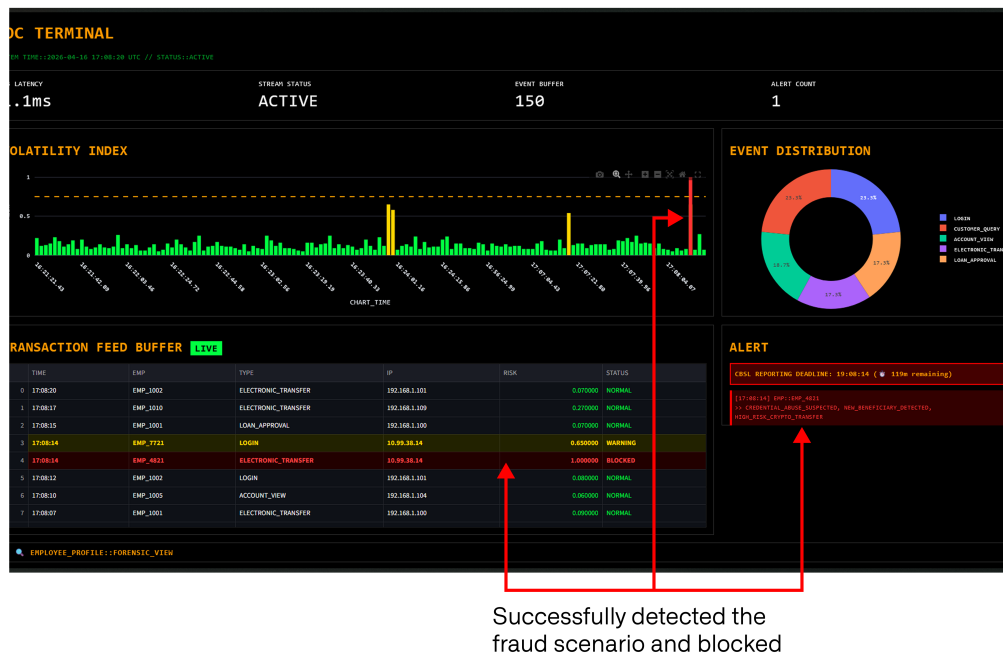


Figure 6.1: Successful fraud detection

The results indicate that the proposed rule engine successfully identified predefined fraud activities while allowing normal employee behavior to continue without generating unnecessary alerts.

6.3 System Latency

System latency represents the delay between event generation and appearance within the monitoring dashboard. System latency was measured by recording the elapsed time between event generation and successful appearance within the monitoring dashboard. Timing measurements were collected separately for database insertion, rule evaluation, and dashboard refresh operations.

The average values shown in Table 6.3 represent results collected across 100 repeated executions. The following stages contribute to total response time:

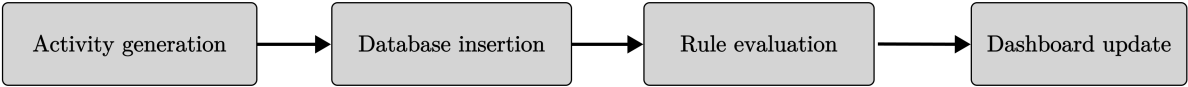


Figure 6.2: Stages contribute to total response timen

Response time observations obtained during testing are shown in Table 6.3.

Table 6.3: Observed response times	
Operation	Average Time
Database insertion	20 ms
Rule evaluation	15 ms
Dashboard refresh	1000–2000 ms
Total response time	1–2 seconds

The largest delay was introduced by dashboard refresh intervals rather than database processing operations. Database insertion and rule evaluation completed within milliseconds, while dashboard rendering introduced additional delay because Streamlit updates visual components periodically. Therefore, the implemented detection process itself operates significantly faster than the visible dashboard response time. Since the dashboard currently refreshes every two seconds, detection itself occurs much faster than visual presentation.

6.4 Scalability Analysis

Scalability analysis was performed to observe system behavior as the number of generated events increased. The system continuously inserted employee activities into MongoDB while the monitoring dashboard retrieved recent records for visualization. Scalability experiments were conducted by gradually increasing the number of generated employee activity records and observing system behavior. Event volumes were incrementally increased from 500 to 5,000 generated records while monitoring dashboard responsiveness and application memory usage. Observations were collected during continuous system execution.

Table 6.4 shows the observed system behavior.

Table 6.4: Scalability observations		
Number of Events	System Behavior	Dashboard Status
500	Stable	Responsive
1000	Stable	Responsive
2000	Stable	Responsive
5000	Minor delay observed	Responsive

The results indicate that the system remained operational as event volumes increased. Small delays were observed at larger workloads, mainly due to dashboard rendering and repeated retrieval operations rather than database limitations.

6.5 Chapter Summary

This chapter evaluated the performance of the proposed insider threat detection system. The experimental setup, detection performance, latency measurements, and scalability behavior were examined. The results showed that the developed implementation successfully detected predefined fraud scenarios while maintaining near real-time performance. The next chapter discusses limitations, trade-offs, and possible future improvements.

Chapter 7

Limitations

7.1 Consistency Trade-offs

One limitation of NoSQL databases involves consistency behavior. Traditional relational databases are designed around strong ACID properties which guarantee that all operations remain completely consistent. MongoDB supports transactions, but large distributed environments may introduce eventual consistency depending on system configuration. For banking environments, this can create challenges because financial systems often require strict consistency guarantees for account balances and transaction records. Therefore, although MongoDB provides advantages for monitoring and fraud detection, core banking transaction systems may still depend on relational databases. The current implementation only focuses on monitoring activities and does not directly modify financial ledger information.

7.2 Storage Overhead

Document-oriented databases often duplicate information across documents to reduce query complexity and eliminate JOIN operations. For example, employee details may appear repeatedly inside multiple activity records:

Listing 7.1: Repeated employee information in documents

```
1 {  
2   "employeeId": "EMP_1002",  
3   "department": "TREASURY",  
4   "actionType": "LOGIN"  
5 }  
6  
7 {  
8   "employeeId": "EMP_1002",  
9   "department": "TREASURY",  
10  "actionType": "TRANSFER"  
11 }
```

Although this approach improves retrieval speed, it increases storage requirements because similar information can appear multiple times. For large banking systems handling millions of activities each day, storage consumption may become an important consideration.

7.3 Security Concerns

NoSQL systems introduce several security challenges that require careful configuration. If applications directly accept user inputs without validation, NoSQL injection attacks may occur. In addition, weak access control configurations could expose sensitive banking information. The current implementation was developed as a research prototype and therefore does not include advanced security mechanisms such as:

- role-based access control,

- encryption at rest,
- multi-factor authentication,
- audit log protection.

These security controls would be necessary in a production environment.

7.4 Operational Challenges

Managing NoSQL systems may require additional operational effort. Performance often depends on proper indexing strategies and database configuration. Incorrect index selection can reduce query performance and increase resource usage. The developed system also uses periodic dashboard refresh operations:

Listing 7.2: Dashboard refresh mechanism

```
1 time.sleep(2)
2 st.rerun()
```

Although this approach creates a near real-time experience, repeated refresh operations may introduce additional processing overhead as the number of events increases. Future implementations could replace this mechanism with database change streams to achieve more efficient real-time updates.

7.5 Future Improvements

Several improvements can be introduced in future versions of the system. Machine learning techniques could be integrated to automatically learn employee behavior patterns rather than relying only on predefined rules. Graph-based analysis may also improve the identification of collusion activities between employees and suspicious accounts. Additional improvements may include:

- real-time MongoDB change streams,
- predictive anomaly detection models,
- graph database integration,
- immutable audit logging.

These enhancements could improve detection accuracy and support more complex fraud scenarios.

7.6 Chapter Summary

This chapter discussed limitations associated with the proposed system and the selected NoSQL approach. Issues related to consistency, storage requirements, security, and operational complexity were identified. Although MongoDB provides flexibility and real-time capabilities, trade-offs still exist and must be considered when deploying systems within financial environments. Future improvements were also identified to enhance scalability and detection capability.

Chapter 8

Conclusion

8.1 Summary of Findings

This project examined the limitations of traditional relational database systems for real-time fraud monitoring and explored how a NoSQL-based approach can improve insider threat detection within banking environments. The study identified several challenges associated with traditional SQL systems including fixed schemas, complex JOIN operations, and delayed monitoring approaches. These limitations become increasingly difficult to manage when fraud patterns continuously evolve and large numbers of banking activities must be processed in real time. To address these issues, a MongoDB-based monitoring system was designed and implemented. The developed system combined synthetic banking data generation, rule-based anomaly detection, real-time activity processing, and a monitoring dashboard capable of visualizing suspicious behavior. The implemented system successfully detected predefined fraud scenarios such as after-hours activities, credential misuse, suspicious beneficiaries, and ghost account behavior. The evaluation results demonstrated that flexible document structures and real-time event handling can support faster monitoring and easier system expansion.

8.2 Benefits for Banking Environments

The proposed system can provide several practical benefits for banking environments where rapid identification of suspicious behavior is important. One benefit is the ability to identify abnormal activities while events are occurring rather than several hours later. Early detection allows security teams to investigate suspicious actions before transactions are fully completed. System also provides flexibility when new fraud indicators appear. Since attack techniques continuously evolve, banks require monitoring systems that can adapt quickly without redesigning database structures or interrupting operations. Another advantage is improved monitoring visibility. The dashboard provides a centralized interface where analysts can observe employee activities, risk scores, and generated alerts in real time. The proposed implementation is not intended to replace core banking transaction systems. Instead, it can operate alongside existing systems as an additional monitoring and surveillance layer to support security operations.

8.3 Final Reflection

Insider threats and financial fraud continue to become more complex as banking systems increasingly depend on digital technologies. Traditional monitoring approaches may struggle to react quickly when attack patterns change rapidly. This project demonstrated how modern database technologies and real-time processing techniques can support more flexible monitoring environments. While the current implementation represents a prototype system, the overall approach illustrates how real-time monitoring can improve security awareness and reduce response delays. The main objective of this work was not to suggest that NoSQL databases completely replace traditional relational systems. Instead, the findings indicate that both technologies can work together depending on the problem being addressed. Developed system demonstrates that combining flexible database structures with real-time detection mechanisms can provide stronger support for identifying suspicious activities and improving banking security operations.

References

- Ambler, S.W. (2003) *Mapping Objects to Relational Databases*. New York: AmbySoft Inc.
- Cappelli, D., Moore, A.P. and Trzeciak, R.F. (2012) *The CERT Guide to Insider Threats: Faced by Information Technology, Systems, and Organizations*. Boston: Addison-Wesley.
- Committee on Public Finance (CoPF) (2026) *Fraud concerns regarding NDB Bank and governance oversight*. Colombo: Parliament of Sri Lanka.
- Daily FT (2026) 'NDB Bank says detected employee-linked fraud', *Daily FT*, 15 May.
- Elmasri, R. and Navathe, S.B. (2017) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
- Greitzer, F.L. and Hohimer, R.E. (2011) 'Modeling human behavior to anticipate insider attacks', *Journal of Strategic Security*, 4(2), pp. 25–48.
- Han, J., Haihong, E., Le, G. and Du, J. (2011) 'Survey on NoSQL database', *Proceedings of the 6th International Conference on Pervasive Computing and Applications*, pp. 363–366.
- MongoDB Inc. (2024) *MongoDB Documentation*. Available at: <https://www.mongodb.com/docs/> (Accessed: 21 May 2026).
- NDB Bank (2026) *Official statement and progress update following fraud incident*. Colombo: National Development Bank PLC.
- Rashid, A., Agrafiotis, I. and Nurse, J.R.C. (2022) 'Insider threat detection in financial systems', *Computers & Security*, 114, p. 102118.
- Sadalage, P.J. and Fowler, M. (2012) *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Boston: Addison-Wesley.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database System Concepts*. 7th edn. New York: McGraw-Hill.
- Sirawongphatsara, P., Pornpongtechavanich, P., Sriamorntrakul, P. and Daengsi, T. (2023) 'Analyzing bank account information of nominees and scammers', *arXiv* preprint arXiv:2308.12345.
- Uddin, M. (2026) 'A regulatory governance framework for AI-driven financial fraud detection in U.S. banking', *arXiv* preprint arXiv:2602.54321.